

ÎNDRUMĂTOR
DIGITAL TWIN + MACHINE LEARNING
+ GRAFANA (WINDOWS) – Continuare
Matlab 2

Contents

ÎNDRUMĂTOR DIGITAL TWIN + MACHINE LEARNING + GRAFANA (WINDOWS) – Continuare Matlab 2.....	1
1. INSTALĂRI NECESARE (ORDINE OBLIGATORIE)	3
1.1 Instalare Docker Desktop (Windows)	3
2. INSTALARE DATABASE + GRAFANA (DOCKER)	3
2.1 Structură de lucru	3
2.2 Docker Compose pentru InfluxDB + Grafana	4
2.3 Pornirea serviciilor	4
3. CONFIGURARE GRAFANA → DATABASE.....	5
4. PARTEA MATLAB – DIGITAL TWIN	5
4.1 Deschiderea proiectului	5
5. GENERAREA DATELOR PENTRU MACHINE LEARNING.....	5
6. MACHINE LEARNING – ANTRENARE MODEL	6
7. GEAMĂN DIGITAL + VERDICT ML	7
8. TRIMITERE DIRECTĂ MATLAB → INFLUXDB.....	7
9. FLUX COMPLET DE DATE (RUN TIME).....	8
10. DASHBOARD GRAFANA – DIGITAL TWIN.....	8
Presiune medie	8
RMS curent.....	8
Alarmă	9
Health Index	9
11. CE SE OBTINE FINAL	9
12. CE SE RAPORTEAZĂ	10

Scop:

Se dezvoltă un geamăn digital al unei pompe în MATLAB/Simscape.

Se simulează defecte, se antrenează un model de Machine Learning pentru evaluarea stării pompei, iar datele sunt trimise într-o bază de date time-series și vizualizate într-o interfață web Grafana.



Tehnologii utilizate (toate gratuite):

- MATLAB + Simulink + Simscape
- Docker Desktop (Windows)
- InfluxDB (database time-series)
- Grafana (dashboard web)

1. INSTALĂRI NECESARE (ORDINE OBLIGATORIE)

1.1 Instalare Docker Desktop (Windows)

1. Se verifică:
 - Windows 10/11 64-bit
 - Virtualizare activată în BIOS
2. Se instalează **Docker Desktop for Windows**
3. Se acceptă utilizarea **WSL2 backend**
4. Se repornește sistemul

Verificare (PowerShell):

```
docker --version  
docker compose version
```

Dacă apar versiuni → Docker este instalat corect.

2. INSTALARE DATABASE + GRAFANA (DOCKER)

2.1 Structură de lucru

Se creează următoarea structură:

```
C:\LabPump\  
├── cloud\  
│   └── docker-compose.yml  
└── Simscape-Triplex-Pump-master\
```

2.2 Docker Compose pentru InfluxDB + Grafana

În C:\LabPump\cloud\docker-compose.yml se scrie:

```
version: "3.8"  
  
services:  
  influxdb:  
    image: influxdb:2  
    ports:  
      - "8086:8086"  
    environment:  
      - DOCKER_INFLUXDB_INIT_MODE=setup  
      - DOCKER_INFLUXDB_INIT_USERNAME=admin  
      - DOCKER_INFLUXDB_INIT_PASSWORD=admin12345  
      - DOCKER_INFLUXDB_INIT_ORG=lab-org  
      - DOCKER_INFLUXDB_INIT_BUCKET=pump_bucket  
      - DOCKER_INFLUXDB_INIT_ADMIN_TOKEN=lab-token-123  
    volumes:  
      - ./influx-data:/var/lib/influxdb2  
  
  grafana:  
    image: grafana/grafana  
    ports:  
      - "3000:3000"  
    volumes:  
      - ./grafana-data:/var/lib/grafana
```

2.3 Pornirea serviciilor

În PowerShell:

```
cd C:\LabPump\cloud  
docker compose up -d
```

Verificare:

```
docker ps
```

Se deschid în browser:

- InfluxDB: <http://localhost:8086>

- Grafana: <http://localhost:3000>

3. CONFIGURARE GRAFANA → DATABASE

1. Se accesează Grafana
2. Login: admin / admin
3. Se schimbă parola
4. Connections → Data sources → Add data source
5. Se selectează **InfluxDB**
6. Se setează:
 - Query language: Flux
 - URL: <http://localhost:8086>
 - Organization: lab-org
 - Token: lab-token-123
 - Default bucket: pump_bucket
7. Save & Test → OK

4. PARTEA MATLAB – DIGITAL TWIN

4.1 Deschiderea proiectului

În MATLAB:

```
cd Simscape-Triplex-Pump-master
openProject('Triplex_Pump.prj')
open_system('sm_pump_triplex.slx')
```

Se verifică rularea modelului fără erori.

5. GENERAREA DATELOR PENTRU MACHINE LEARNING

Se creează fișierul:

```
generate_pump_data_lab.m

N = 20;
features = zeros(N,2);
labels = strings(N,1);
```

```

for k = 1:N
    if rand < 0.5
        faultMode = 'normal';
    else
        faultMode = 'seal_leakage';
    end

    assignin('base','faultMode', faultMode);

    simOut = sim('sm_pump_triplex','ReturnWorkspaceOutputs','on');

    p = simOut.logouts.get('OutletPressure').Values.Data;
    i = simOut.logouts.get('MotorCurrent').Values.Data;

    features(k,1) = mean(p);
    features(k,2) = rms(i);
    labels(k) = faultMode;
end

save('pump_data_lab.mat','features','labels');

```

Se rulează:

```
run('generate_pump_data_lab.m')
```

6. MACHINE LEARNING – ANTRENARE MODEL

Se creează fișierul:

```

train_pump_ml.m

load('pump_data_lab.mat');

X = features;
Y = categorical(labels);

idx = randperm(size(X,1));
nTrain = round(0.7*length(idx));

Xtrain = X(idx(1:nTrain),:);
Ytrain = Y(idx(1:nTrain));

Xtest = X(idx(nTrain+1:end),:);
Ytest = Y(idx(nTrain+1:end));

model = fitctree(Xtrain,Ytrain);

pred = predict(model,Xtest);
acc = mean(pred==Ytest)*100;
disp(['Acuratete: ',num2str(acc),' %']);

save('pump_ml_model.mat','model');

```

7. GEAMĂN DIGITAL + VERDICT ML

Se creează funcția:

```
evaluate_pump_once.m

function res = evaluate_pump_once(faultMode)

persistent model
if isempty(model)
    load('pump_ml_model.mat','model');
end

assignin('base','faultMode',faultMode);

simOut = sim('sm_pump_triplex','ReturnWorkspaceOutputs','on');

p = simOut.logout.get('OutletPressure').Values.Data;
i = simOut.logout.get('MotorCurrent').Values.Data;

res.meanPressure = mean(p);
res.rmsCurrent = rms(i);

res.predictedLabel = string(predict(model,[res.meanPressure
res.rmsCurrent]));
res.faultMode = string(faultMode);

res.healthIndex = double(res.predictedLabel=="normal");
res.alarm = double(res.healthIndex==0);
end
```

8. TRIMITERE DIRECTĂ MATLAB → INFLUXDB

Se creează:

```
push_to_influx.m

function push_to_influx(res)

url = "http://localhost:8086/api/v2/write";
params = struct("org","lab-org","bucket","pump_bucket","precision","ns");
token = "lab-token-123";

ts = int64(posixtime(datetime("now"))*1e9);

line = sprintf( ...
    "pump_status,faultMode=%s,predLabel=%s
meanPressure=%f,rmsCurrent=%f,healthIndex=%f,alarm=%d %d", ...
    res.faultMode,res.predictedLabel,res.meanPressure,res.rmsCurrent,res.healthIndex,res.alarm,ts);
```

```

headers = [
    matlab.net.http.HeaderField("Authorization","Token "+token)
    matlab.net.http.HeaderField("Content-Type","text/plain")
];

req = matlab.net.http.RequestMessage("post",headers,line);
uri = matlab.net.URI(url);
uri.Query = matlab.net.QueryParameter(params);
send(req,uri);
end

```

9. FLUX COMPLET DE DATE (RUN TIME)

Se creează:

```

run_monitoring_to_grafana.m

for k = 1:60
    if rand < 0.8
        fm = "normal";
    else
        fm = "seal_leakage";
    end

    res = evaluate_pump_once(fm);
    push_to_influx(res);

    pause(2);
end

```

Se rulează:

```

run('run_monitoring_to_grafana.m')

```

10. DASHBOARD GRAFANA – DIGITAL TWIN

Se creează dashboard cu următoarele panouri:

Presiune medie

```

from(bucket:"pump_bucket")
  |> range(start:-15m)
  |> filter(fn:(r)=>r._measurement=="pump_status")
  |> filter(fn:(r)=>r._field=="meanPressure")

```

RMS curent

```

from(bucket:"pump_bucket")

```

```
|> range(start:-15m)
|> filter(fn:(r)=>r._measurement=="pump_status")
|> filter(fn:(r)=>r._field=="rmsCurrent")
```

Alarmă

```
from(bucket:"pump_bucket")
|> range(start:-15m)
|> filter(fn:(r)=>r._measurement=="pump_status")
|> filter(fn:(r)=>r._field=="alarm")
|> last()
```

Health Index

```
from(bucket:"pump_bucket")
|> range(start:-15m)
|> filter(fn:(r)=>r._measurement=="pump_status")
|> filter(fn:(r)=>r._field=="healthIndex")
```

11. CE SE OBTINE FINAL

- Geamă digital funcțional
- Simulare defecte
- Clasificare ML automată
- Bază de date time-series
- Vizualizare web live în Grafana



- Fundament pentru mentenanță predictive

12. CE SE RAPORTEAZĂ

- Imagini cu interfața Grafana
- Explicații privind modul de preluare și inserare a informațiilor în Grafana
- Explicarea metodei de antrenare pentru mentenanța predictivă